Globally Converging Algorithm for Multistage Stochastic Mixed-integer Programming

Hanbin Yang

hanbinyang@ust.hk

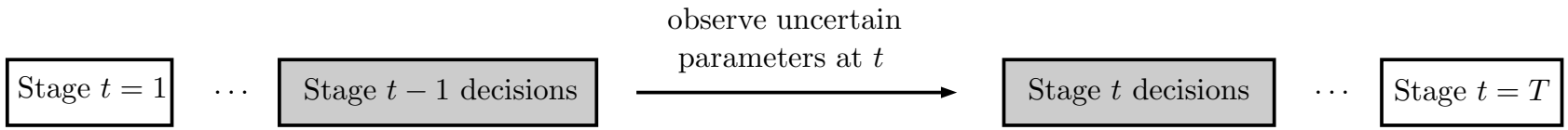
The Hong Kong University of Science and Technology

November 29, 2025 at SUSTech

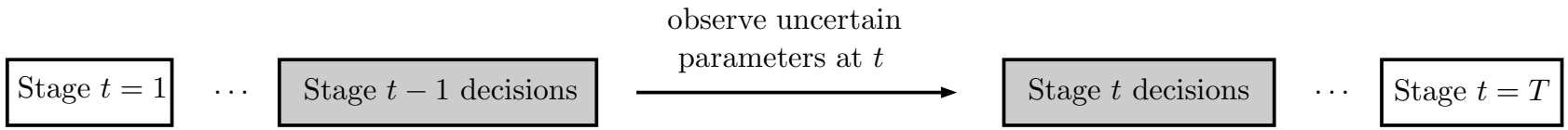


香港科技大學
THE HONG KONG UNIVERSITY OF
SCIENCE AND TECHNOLOGY

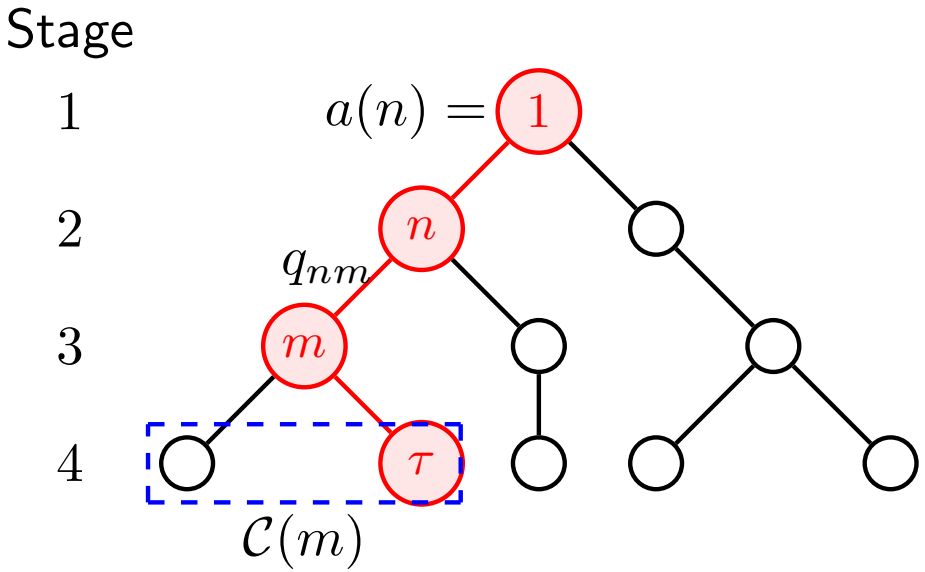
Multistage Stochastic Programming



Multistage Stochastic Programming



Describe the stochastic process $(\tilde{\boldsymbol{\xi}}_1, \ldots, \tilde{\boldsymbol{\xi}}_T)$ via a scenario tree $\mathcal{N}$:



Node 1: root node;
$a(n)$: parent node of n;
q_{nm}: transition probability;
$\mathcal{C}(n)$: set of children nodes of n;
$\mathcal{P}^\omega = \{1, n, m, \tau\}$: scenario path.

Dynamic Programming Formulation

The recursive formulation with expected cost-to-go functions:

$$V^* = \min_{x_1} \quad \overbrace{f_1(x_1)}^{\text{current cost}} + \overbrace{\sum_{m \in \mathcal{C}(1)} q_{1m} Q_m(x_1)}^{\text{future cost}} \qquad \text{(P)}$$

$$\text{s.t.} \quad (x_0, x_1) \in X_1.$$

Dynamic Programming Formulation

The recursive formulation with expected cost-to-go functions:

$$V^* = \min_{x_1} \quad \overbrace{f_1(x_1)}^{\text{current cost}} + \overbrace{\sum_{m \in \mathcal{C}(1)} q_{1m} Q_m(x_1)}^{\text{future cost}} \qquad \text{(P)}$$

$$\text{s.t.} \quad (x_0, x_1) \in X_1.$$

The nodal subproblem at node $n \in \mathcal{N}$ is defined as:

$$Q_n(x_{a(n)}) := \min_{x_n} \quad f_n(x_n) + \sum_{m \in \mathcal{C}(n)} q_{nm} Q_m(x_n)$$

$$\text{s.t.} \quad (x_{a(n)}, x_n) \in X_n.$$

Dynamic Programming Formulation

The recursive formulation with expected cost-to-go functions:

$$V^* = \min_{x_1} \quad \overbrace{f_1(x_1)}^{\text{current cost}} + \overbrace{\sum_{m \in \mathcal{C}(1)} q_{1m} Q_m(x_1)}^{\text{future cost}} \qquad \text{(P)}$$

$$\text{s.t.} \quad (x_0, x_1) \in X_1.$$

The nodal subproblem at node $n \in \mathcal{N}$ is defined as:

$$Q_n(x_{a(n)}) := \min_{x_n} \quad f_n(x_n) + \sum_{m \in \mathcal{C}(n)} q_{nm} Q_m(x_n)$$

$$\text{s.t.} \quad (x_{a(n)}, x_n) \in X_n.$$

Properties of Q_n given linearity of f_n:

- X_n is a Polytope: $Q_n(\cdot)$ is convex and piece-wise linear;

Dynamic Programming Formulation

The recursive formulation with expected cost-to-go functions:

$$V^* = \min_{x_1} \quad \overbrace{f_1(x_1)}^{\text{current cost}} + \overbrace{\sum_{m \in \mathcal{C}(1)} q_{1m} Q_m(x_1)}^{\text{future cost}} \qquad \text{(P)}$$

$$\text{s.t.} \quad (x_0, x_1) \in X_1.$$

The nodal subproblem at node $n \in \mathcal{N}$ is defined as:

$$Q_n(x_{a(n)}) := \min_{x_n} \quad f_n(x_n) + \sum_{m \in \mathcal{C}(n)} q_{nm} Q_m(x_n)$$

$$\text{s.t.} \quad (x_{a(n)}, x_n) \in X_n.$$

Properties of Q_n given linearity of f_n:

- X_n is a Polytope: $Q_n(\cdot)$ is convex and piece-wise linear;
- X_n is a mixed-integer polytope: $Q_n(\cdot)$ is lower semi-continuous (l.s.c.).

Value Function Approximation

- ϕ_m is an under-approximation of Q_m;
- ϕ_m has an explicit formulation that can be evaluated efficiently.

Value Function Approximation

- ϕ_m is an under-approximation of Q_m;
- ϕ_m has an explicit formulation that can be evaluated efficiently.

$$\phi_m(x_n) := \min \left\{ \; \theta_m \;\middle|\; \theta_m \geq v_m^\ell + {\pi_m^\ell}^\top x_n, \quad \forall \ell = 0, \ldots, L_m \; \right\}$$

Value Function Approximation

- ϕ_m is an under-approximation of Q_m;
- ϕ_m has an explicit formulation that can be evaluated efficiently.

$$\phi_m(x_n) := \min \left\{ \theta_m \;\middle|\; \theta_m \geq v_m^\ell + {\pi_m^\ell}^\top x_n, \quad \forall \ell = 0, \ldots, L_m \right\}$$

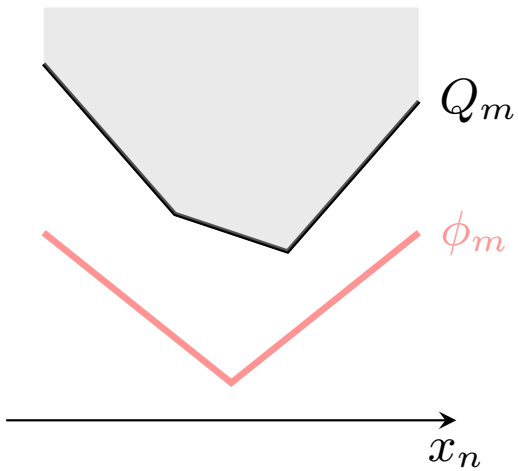


Figure: Q_m is convex.

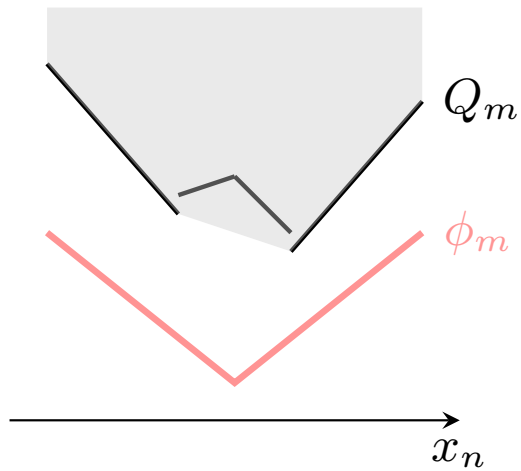


Figure: Q_m is l.s.c.

Approximate Nodal Subproblem

Approximate $\{Q_m\}_{m\in\mathcal{C}(n)}$ by cut approximators $\{\phi_m\}_{m\in\mathcal{C}(n)}$:

$$
\begin{aligned}
&\underline{Q}_n(x_{a(n)}) := \\
&\quad \min \quad f_n(x_n) + \sum_{m\in\mathcal{C}(n)} q_{nm} \cancel{Q_m(x_n)} \phi_m(x_n) && (P_n) \\
&\quad \text{s.t.} \quad (\cancel{x_{a(n)}}\, z_n, x_n) \in X_n \\
&\qquad\quad z_n = x_{a(n)} && (\text{NA})
\end{aligned}
$$

where z_n is the copy variable with the **non-anticipativity** constraint.

Approximate Nodal Subproblem

Approximate $\{Q_m\}_{m\in\mathcal{C}(n)}$ by cut approximators $\{\phi_m\}_{m\in\mathcal{C}(n)}$:

$$
\begin{aligned}
&\underline{Q}_n(x_{a(n)}) := \\
&\quad \min \quad f_n(x_n) + \sum_{m\in\mathcal{C}(n)} q_{nm} \cancel{Q_m(x_n)} \phi_m(x_n) && (P_n) \\
&\quad \text{s.t.} \quad (\cancel{x_{a(n)}}\ z_n, x_n) \in X_n \\
&\qquad\quad z_n = x_{a(n)} && (\text{NA})
\end{aligned}
$$

where z_n is the copy variable with the **non-anticipativity** constraint.

- $\underline{Q}_n(\cdot)$ under-approximates the true value function Q_n;

Approximate Nodal Subproblem

Approximate $\{Q_m\}_{m\in\mathcal{C}(n)}$ by cut approximators $\{\phi_m\}_{m\in\mathcal{C}(n)}$:

$$\underline{Q}_n(x_{a(n)}) :=$$

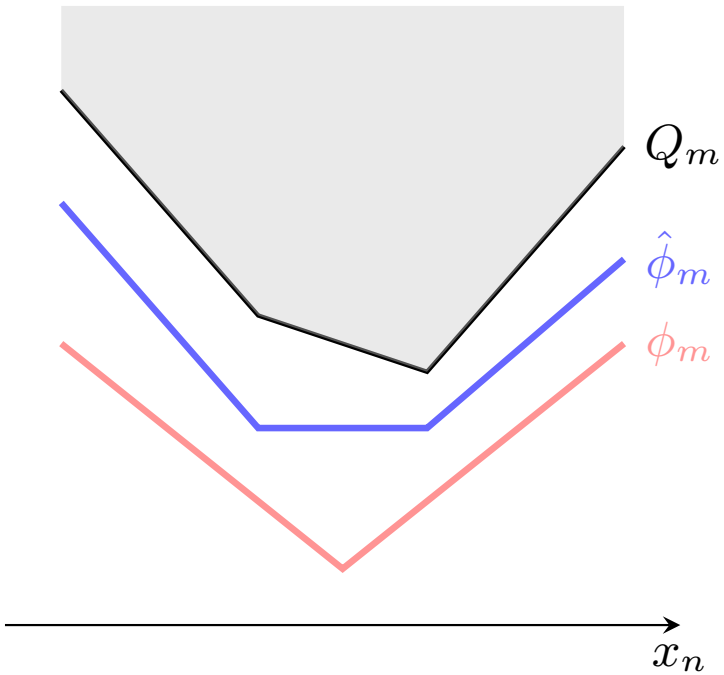
$$\begin{aligned} \min \quad & f_n(x_n) + \sum_{m\in\mathcal{C}(n)} q_{nm} \cancel{Q_m(x_n)} \phi_m(x_n) && (P_n) \\ \text{s.t.} \quad & (\cancel{x_{a(n)}}\, z_n, x_n) \in X_n \\ & z_n = x_{a(n)} && \text{(NA)} \end{aligned}$$

where z_n is the copy variable with the **non-anticipativity** constraint.

- $\underline{Q}_n(\cdot)$ under-approximates the true value function Q_n;
- Constraint (NA) is used to generate cuts that improve the approximation.

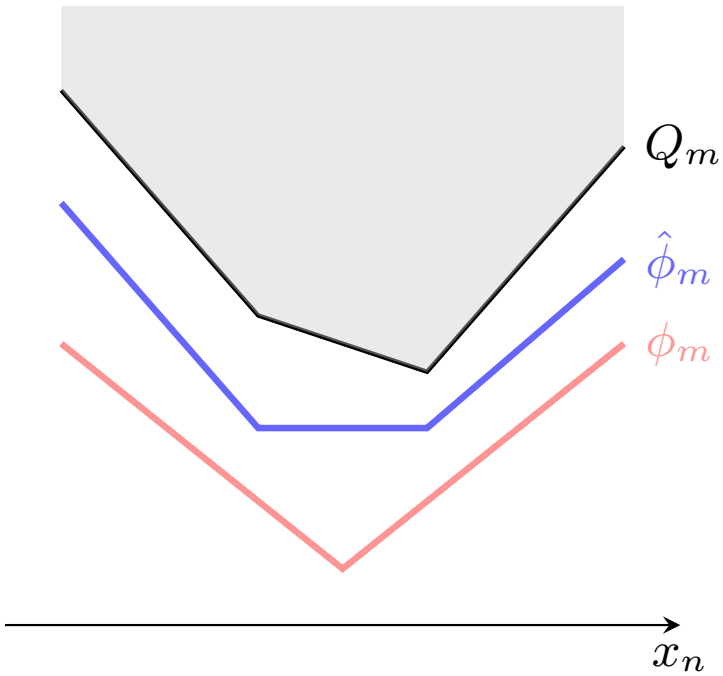
Refinement of Cut Approximator ϕ_m

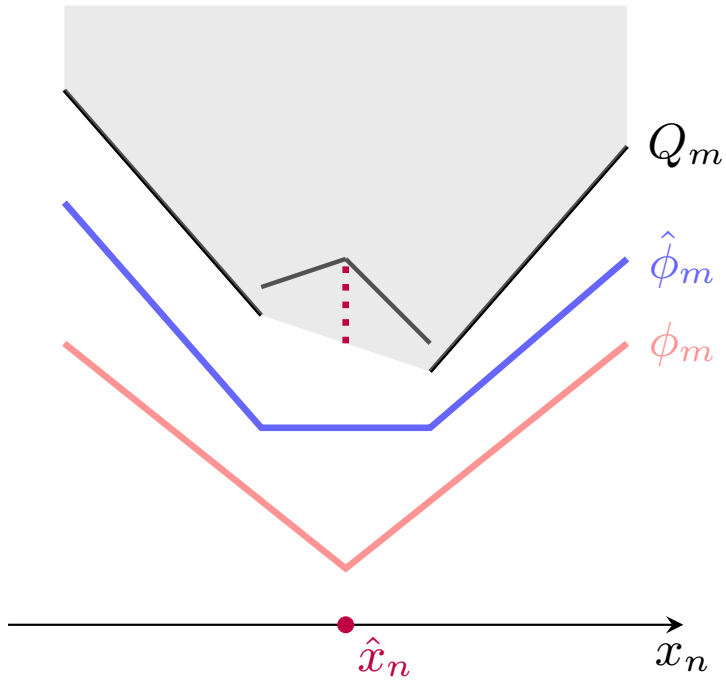
- ϕ_m is improved by incorporating cuts $\rightsquigarrow$ $\hat{\phi}_m$;



Refinement of Cut Approximator ϕ_m

- ϕ_m is improved by incorporating cuts $\rightsquigarrow$ $\hat{\phi}_m$;





- If Q_m is l.s.c., there always exists gap between ϕ_m and Q_m.

Definition

Let f be an l.s.c. function. The *epigraph* of f over $\mathcal{Z}$ is

$$\mathrm{epi}_{\mathcal{Z}}(f) = \{(x, \eta) \mid \eta \geq f(x)\ \forall x \in \mathcal{Z}\}.$$

The *convex envelope* of f over $\mathcal{Z}$ is

$$\mathrm{co}_{\mathcal{Z}}(f)(x) = \min\{\eta \mid (x, \eta) \in \mathrm{conv}(\mathrm{epi}_{\mathcal{Z}}(f))\}.$$

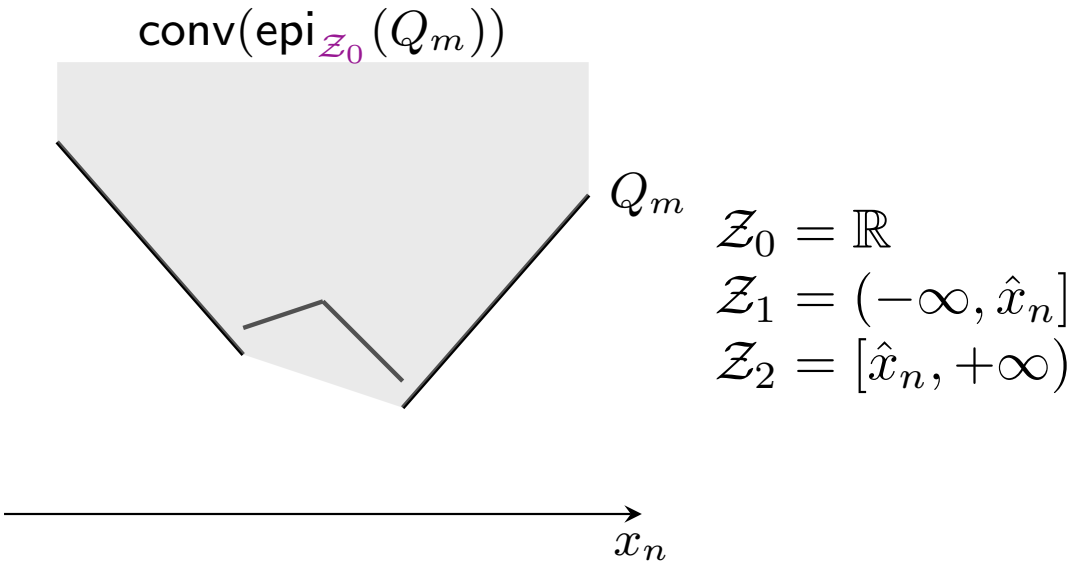
Definition

Let f be an l.s.c. function. The *epigraph* of f over $\mathcal{Z}$ is

$$\text{epi}_{\mathcal{Z}}(f) = \{(x, \eta) \mid \eta \geq f(x)\ \forall x \in \mathcal{Z}\}.$$

The *convex envelope* of f over $\mathcal{Z}$ is

$$\text{co}_{\mathcal{Z}}(f)(x) = \min\{\eta \mid (x, \eta) \in \text{conv}(\text{epi}_{\mathcal{Z}}(f))\}.$$



$$\mathcal{Z}_0 = \mathbb{R}$$
$$\mathcal{Z}_1 = (-\infty, \hat{x}_n]$$
$$\mathcal{Z}_2 = [\hat{x}_n, +\infty)$$

Figure: Convex envelope over $\mathcal{Z}_0$.

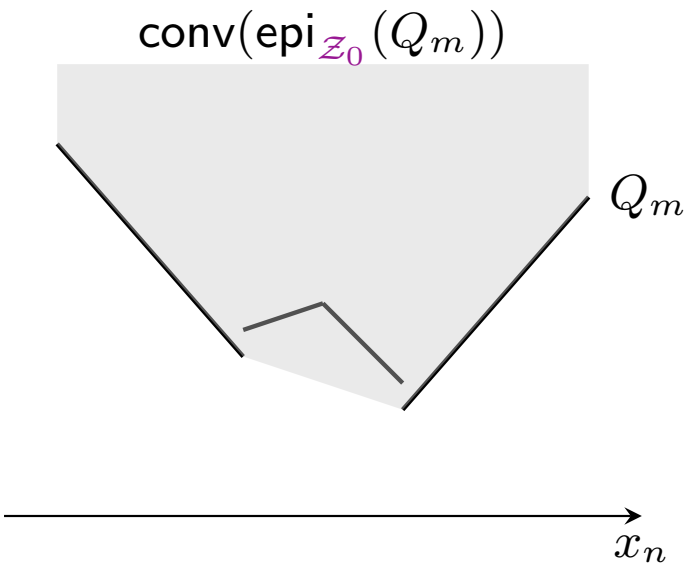
Definition

Let f be an l.s.c. function. The *epigraph* of f over $\mathcal{Z}$ is

$$\text{epi}_{\mathcal{Z}}(f) = \{(x, \eta) \mid \eta \geq f(x)\ \forall x \in \mathcal{Z}\}.$$

The *convex envelope* of f over $\mathcal{Z}$ is

$$\text{co}_{\mathcal{Z}}(f)(x) = \min\{\eta \mid (x, \eta) \in \text{conv}(\text{epi}_{\mathcal{Z}}(f))\}.$$



$$\mathcal{Z}_0 = \mathbb{R}$$
$$\mathcal{Z}_1 = (-\infty, \hat{x}_n]$$
$$\mathcal{Z}_2 = [\hat{x}_n, +\infty)$$

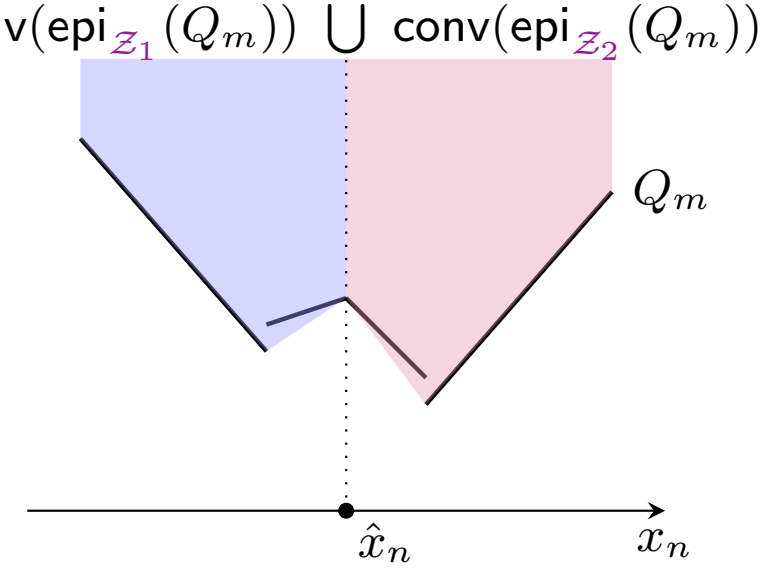


Figure: Convex envelope over $\mathcal{Z}_0$.

Figure: **Piecewise** convex envelope.

Lagrangian Cuts & Properties

Lagrangian-Cut Generation Program

Goal: Refine ϕ_m by adding new cuts.

Given: State $\hat{x}_n$ at parent node $n = a(m)$.

The Lagrangian dual problem is:

$$\max_{\pi_m} \quad \mathcal{L}_m(\pi_m), \tag{D_m}$$

where $\mathcal{L}_m(\pi_m) :=$

$$\begin{aligned} \min \quad & f_m(x_m) + \sum_{\tau \in \mathcal{C}(m)} q_{m\tau}\phi_\tau(x_m) + \pi_m^\top(\hat{x}_n - z_m) \\ \text{s.t.} \quad & (z_m, x_m) \in X_m \\ & z_m \in \mathcal{Z}_n \end{aligned}$$

Note: Restrict z_m into a predefined region $\mathcal{Z}_n$.

Lagrangian Cuts & Properties

Given: An optimal dual solution π_m^* to (D_m).

Generate a Lagrangian cut:

$$\theta_m \geq v_m^* + {\pi_m^*}^\top x_n,$$

where $v_m^* := \mathcal{L}_m(\pi_m^*) - {\pi_m^*}^\top \hat{x}_n$.

Lagrangian Cuts & Properties

Given: An optimal dual solution π_m^* to (D_m).

Generate a Lagrangian cut:

$$\theta_m \geq v_m^* + {\pi_m^*}^\top x_n,$$

where $v_m^* := \mathcal{L}_m(\pi_m^*) - {\pi_m^*}^\top \hat{x}_n$.

The Lagrangian cut $\theta_m \geq v_m^* + {\pi_m^*}^\top x_n$ is:

1. valid over $\mathcal{Z}_n$: $\forall\, x_n \in \mathcal{Z}_n$,

$$Q_m(x_n) \geq \underline{Q}_m(x_n) \geq v_m^* + {\pi_m^*}^\top x_n;$$

2. tight at $\hat{x}_n$ for the convex envelope of $\underline{Q}_m$ over $\mathcal{Z}_n$:

$$v_m^* + {\pi_m^*}^\top \hat{x}_n = \mathrm{co}_{\mathcal{Z}_n}\left(\underline{Q}_m\right)(\hat{x}_n)\,.$$

Outline

1 Introduction

2 Lift-and-Cut Algorithm

3 Cut Enhancement

4 Numerical Experiments

5 Summary

From Lagrangian Cuts to Global Optimality

Motivation:

- Lagrangian cuts recover the convex envelope of $\underline{Q}_m$ over $\mathcal{Z}_n$;
- This provides an under-approximation of Q_m over $\mathcal{Z}_n$;

From Lagrangian Cuts to Global Optimality

Motivation:

- Lagrangian cuts recover the convex envelope of $\underline{Q}_m$ over $\mathcal{Z}_n$;
- This provides an under-approximation of Q_m over $\mathcal{Z}_n$;

$\rightsquigarrow$ Goal: Piecewise convex envelopes.

From Lagrangian Cuts to Global Optimality

Motivation:

- Lagrangian cuts recover the convex envelope of $\underline{Q}_m$ over $\mathcal{Z}_n$;
- This provides an under-approximation of Q_m over $\mathcal{Z}_n$;

$\rightsquigarrow$ Goal: Piecewise convex envelopes.

Key Insight:

- Partition domain into smaller subregions:
  - Use binary indicator variables to indicate the selected regions;

From Lagrangian Cuts to Global Optimality

Motivation:

- Lagrangian cuts recover the convex envelope of $\underline{Q}_m$ over $\mathcal{Z}_n$;
- This provides an under-approximation of Q_m over $\mathcal{Z}_n$;

$\rightsquigarrow$ Goal: Piecewise convex envelopes.

Key Insight:

- Partition domain into smaller subregions:
  - Use binary indicator variables to indicate the selected regions;
- Generate Lagrangian cuts in the lifted space.

Partition State Space

Step 1: Partition the State Space

- Each state variable $x_{n,j}$ lies in an interval $[\underline{x}_{n,j}, \overline{x}_{n,j}]$;
- We divide this interval into smaller subintervals:

$$\mathcal{P}_{n,j} := \left\{ [\underline{x}^{\sigma}_{n,j}, \overline{x}^{\sigma}_{n,j}] \ : \ \sigma \in \mathcal{S}_{n,j} \right\}.$$

Partition State Space

Step 1: Partition the State Space

- Each state variable $x_{n,j}$ lies in an interval $[\underline{x}_{n,j}, \overline{x}_{n,j}]$;
- We divide this interval into smaller subintervals:

$$\mathcal{P}_{n,j} := \left\{ [\underline{x}^{\sigma}_{n,j}, \overline{x}^{\sigma}_{n,j}] \,:\, \sigma \in \mathcal{S}_{n,j} \right\}.$$

Step 2: Add Binary Indicators

- For each $\sigma \in \mathcal{S}_{n,j}$, introduce a binary variable $\lambda^{\sigma}_{n,j} \in \{0, 1\}$;
- $\lambda^{\sigma}_{n,j} = 1$ means that $x_{n,j}$ lies in interval $[\underline{x}^{\sigma}_{n,j}, \overline{x}^{\sigma}_{n,j}]$.

Partition State Space

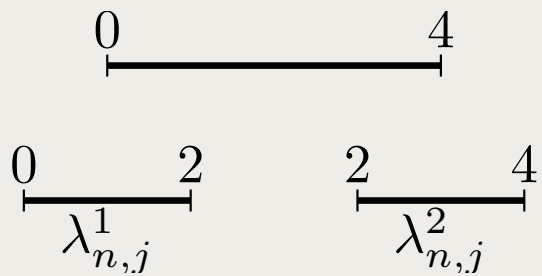
Step 1: Partition the State Space

- Each state variable $x_{n,j}$ lies in an interval $[\underline{x}_{n,j}, \overline{x}_{n,j}]$;
- We divide this interval into smaller subintervals:

$$\mathcal{P}_{n,j} := \left\{ [\underline{x}^{\sigma}_{n,j}, \overline{x}^{\sigma}_{n,j}] \ : \ \sigma \in \mathcal{S}_{n,j} \right\}.$$

Step 2: Add Binary Indicators

- For each $\sigma \in \mathcal{S}_{n,j}$, introduce a binary variable $\lambda^{\sigma}_{n,j} \in \{0, 1\}$;
- $\lambda^{\sigma}_{n,j} = 1$ means that $x_{n,j}$ lies in interval $[\underline{x}^{\sigma}_{n,j}, \overline{x}^{\sigma}_{n,j}]$.



Let $x_{n,j}$ be continuous over $[0, 4]$.

- $\mathcal{P}_{n,j} = \{[0, 2], [2, 4]\}$.
- $\lambda^1_{n,j} = 1$ if $x_{n,j} \in [0, 2]$;
- $\lambda^2_{n,j} = 1$ if $x_{n,j} \in [2, 4]$.

Logical Constraints

Step 3: Logical Constraints

- Ensure each $x_{n,j}$ is assigned to exactly one interval:

$$\sum_{\sigma \in \mathcal{S}_{n,j}} \lambda_{n,j}^{\sigma} = 1.$$

- Enforce consistency between $x_{n,j}$ and $\lambda_{n,j}^{\sigma}$:

$$\sum_{\sigma \in \mathcal{S}_{n,j}} \underline{x}_{n,j}^{\sigma} \lambda_{n,j}^{\sigma} \leq x_{n,j} \leq \sum_{\sigma \in \mathcal{S}_{n,j}} \overline{x}_{n,j}^{\sigma} \lambda_{n,j}^{\sigma}.$$

Logical Constraints

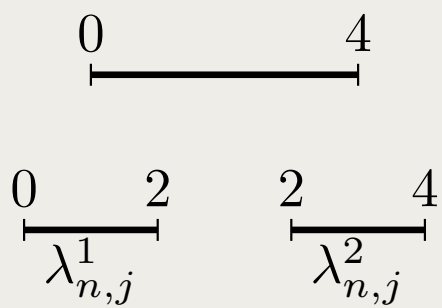
Step 3: Logical Constraints

- Ensure each $x_{n,j}$ is assigned to exactly one interval:

$$\sum_{\sigma \in \mathcal{S}_{n,j}} \lambda_{n,j}^{\sigma} = 1.$$

- Enforce consistency between $x_{n,j}$ and $\lambda_{n,j}^{\sigma}$:

$$\sum_{\sigma \in \mathcal{S}_{n,j}} \underline{x}_{n,j}^{\sigma} \lambda_{n,j}^{\sigma} \leq x_{n,j} \leq \sum_{\sigma \in \mathcal{S}_{n,j}} \overline{x}_{n,j}^{\sigma} \lambda_{n,j}^{\sigma}.$$



$$\lambda_{n,j} \in \{0,1\}^2$$

$$\lambda_{n,j}^1 + \lambda_{n,j}^2 = 1$$

$$0 \cdot \lambda_{n,j}^1 + 2 \cdot \lambda_{n,j}^2 \leq x_{n,j} \leq 2 \cdot \lambda_{n,j}^1 + 4 \cdot \lambda_{n,j}^2$$

Lifted Space

Lift the x_n space to x_n-λ_n space:

- The full partition $\mathbb{P}_n := \prod_j \mathcal{P}_{n,j}$ forms a grid of boxes;

Lifted Space

Lift the x_n space to x_n-λ_n space:

- The full partition $\mathbb{P}_n := \prod_j \mathcal{P}_{n,j}$ forms a grid of boxes;
- (x_n, λ_n) is a valid lifted state if $\lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)$:

$$\Lambda_n(x_n, \mathbb{P}_n) := \left\{ \lambda_n \;\middle|\; \begin{array}{l} \lambda^\sigma_{n,j} \in \{0,1\} \quad \forall \sigma \in \mathcal{S}_{n,j} \\ \sum_{\sigma \in \mathcal{S}_{n,j}} \lambda^\sigma_{n,j} = 1 \\ \sum_{\sigma \in \mathcal{S}_{n,j}} \underline{x}^\sigma_{n,j} \lambda^\sigma_{n,j} \le x_{n,j} \le \sum_{\sigma \in \mathcal{S}_{n,j}} \overline{x}^\sigma_{n,j} \lambda^\sigma_{n,j} \end{array} , \; \forall j \right\}.$$

Lifted Space

Lift the x_n space to x_n-λ_n space:

- The full partition $\mathbb{P}_n := \prod_j \mathcal{P}_{n,j}$ forms a grid of boxes;
- (x_n, λ_n) is a valid lifted state if $\lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)$:

$$\Lambda_n(x_n, \mathbb{P}_n) := \left\{ \lambda_n \;\middle|\; \begin{array}{l} \lambda^\sigma_{n,j} \in \{0,1\} \quad \forall \sigma \in \mathcal{S}_{n,j} \\ \sum_{\sigma \in \mathcal{S}_{n,j}} \lambda^\sigma_{n,j} = 1 \\ \sum_{\sigma \in \mathcal{S}_{n,j}} \underline{x}^\sigma_{n,j} \lambda^\sigma_{n,j} \le x_{n,j} \le \sum_{\sigma \in \mathcal{S}_{n,j}} \overline{x}^\sigma_{n,j} \lambda^\sigma_{n,j} \end{array} , \; \forall j \right\}.$$

In the next few slides, I will explain:

- How to approximate Q_m in the lifted space;
- Why lifting enables tighter, piecewise convex-envelope approximations.

Cutting-plane Approximation

In x_n-λ_n space, the cut approximator is defined as:

$$\phi_m^L(x_n, \lambda_n) := \min \left\{ \theta_m \;\middle|\; \theta_m \geq v_m^\ell + {\pi_m^\ell}^\top x_n + {\rho_m^\ell}^\top \lambda_n, \quad \forall \ell = 0, \ldots, L_m \right\}.$$

Cutting-plane Approximation

In x_n-λ_n space, the cut approximator is defined as:

$$\phi_m^L(x_n, \lambda_n) := \min \left\{ \theta_m \;\middle|\; \theta_m \geq v_m^\ell + {\pi_m^\ell}^\top x_n + {\rho_m^\ell}^\top \lambda_n, \quad \forall \ell = 0, \ldots, L_m \right\}.$$

Define $\underline{Q}_n^L(x_{a(n)}, \lambda_{a(n)}) :=$

$$\begin{aligned} \min \quad & f_n(x_n) + \sum_{m \in \mathcal{C}(n)} q_{nm} \cancel{Q_m(x_n)} \phi_m^L(x_n, \lambda_n) && (P_n^L) \\ \text{s.t.} \quad & (z_n, x_n) \in X_n \\ & \lambda_n \in \Lambda_n(x_n, \mathbb{P}_n) \\ & z_n = x_{a(n)}, \ \mu_n = \lambda_{a(n)}. && \text{(Non-Anticipativity)} \end{aligned}$$

- $\lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)$ ensures that λ_n are consistent with the partitions $\mathbb{P}_n$.

Lifting the Lagrangian Dual Problem

Given: A lifted state $(\hat{x}_n, \hat{\lambda}_n)$ at node n.

Define the lifted Lagrangian dual problem at node $m \in \mathcal{C}(n)$ as:

$$\max_{\pi_m, \rho_m} \quad \mathcal{L}_m^L(\pi_m, \rho_m), \tag{D_m^L}$$

where $\mathcal{L}_m^L(\pi_m, \rho_m) :=$

$$\begin{aligned} \min \quad & f_m(x_m) + \sum_{\tau \in \mathcal{C}(m)} q_{m\tau} \phi_\tau^L(x_m, \lambda_m) \\ & + \pi_m^\top(\hat{x}_n - z_m) + \rho_m^\top(\hat{\lambda}_n - \mu_m) \\ \text{s.t.} \quad & (z_m, x_m) \in X_m \\ & \lambda_m \in \Lambda_m(x_m, \mathbb{P}_m) \\ & z_m \in \mathcal{Z}_n, \mu_m \in \Lambda_n(z_m, \mathbb{P}_n) \end{aligned}$$

Lifted Lagrangian Cut

Given: An optimal dual solution (π_m^*, ρ_m^*) to (D_m^L).

Generate a lifted Lagrangian cut:

$$\theta_m \geq v_m^* + {\pi_m^*}^\top x_n + {\rho_m^*}^\top \lambda_n.$$

where $v_m^* = \mathcal{L}_m^L(\pi_m^*, \rho_m^*) - {\pi_m^*}^\top \hat{x}_n - {\rho_m^*}^\top \hat{\lambda}_n$.

Lifted Lagrangian Cut

Given: An optimal dual solution (π_m^*, ρ_m^*) to (D_m^L).

Generate a lifted Lagrangian cut:

$$\theta_m \geq v_m^* + {\pi_m^*}^\top x_n + {\rho_m^*}^\top \lambda_n.$$

where $v_m^* = \mathcal{L}_m^L(\pi_m^*, \rho_m^*) - {\pi_m^*}^\top \hat{x}_n - {\rho_m^*}^\top \hat{\lambda}_n$.

What are the properties of the lifted Lagrangian cuts?

Properties of Lifted Lagrangian Cuts

Define the restricted region in the lifted space (counterpart of $\mathcal{Z}_n$):

$$\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n) := \{(x_n, \lambda_n) \mid x_n \in \mathcal{Z}_n, \lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)\}.$$

Properties of Lifted Lagrangian Cuts

Define the restricted region in the lifted space (counterpart of $\mathcal{Z}_n$):

$$\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n) := \{(x_n, \lambda_n) \mid x_n \in \mathcal{Z}_n, \lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)\}.$$

Then the lifted Lagrangian cut

$$\theta_m \geq v_m^* + (\pi_m^*)^\top x_n + (\rho_m^*)^\top \lambda_n$$

is:

1. **valid** over $\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)$, i.e., $\forall (x_n, \lambda_n) \in \mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)$

$$Q_m(x_n) \geq \underline{Q}_m^L(x_n, \lambda_n) \geq v_m^* + (\pi_m^*)^\top x_n + (\rho_m^*)^\top \lambda_n;$$

Properties of Lifted Lagrangian Cuts

Define the restricted region in the lifted space (counterpart of $\mathcal{Z}_n$):

$$\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n) := \{(x_n, \lambda_n) \mid x_n \in \mathcal{Z}_n, \lambda_n \in \Lambda_n(x_n, \mathbb{P}_n)\}.$$

Then the lifted Lagrangian cut

$$\theta_m \geq v_m^* + (\pi_m^*)^\top x_n + (\rho_m^*)^\top \lambda_n$$

is:

1. **valid** over $\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)$, i.e., $\forall (x_n, \lambda_n) \in \mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)$

$$Q_m(x_n) \geq \underline{Q}_m^L(x_n, \lambda_n) \geq v_m^* + (\pi_m^*)^\top x_n + (\rho_m^*)^\top \lambda_n;$$

2. **tight** for the convex envelope over $\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)$ at incumbent $(\hat{x}_n, \hat{\lambda}_n)$:

$$\mathcal{L}_m^L(\pi_m^*, \rho_m^*) = \mathrm{co}_{\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n)} \left(\underline{Q}_m^L\right)(\hat{x}_n, \hat{\lambda}_n);$$

3. **tight** in original space:

   The active region for the lifted state $(\hat{x}_n, \hat{\lambda}_n)$ is:

$$\hat{\mathcal{Z}}_n := \mathcal{Z}_n \cap \prod_j \left[\underline{x}_{n,j}^{\hat{\sigma}_j},\ \overline{x}_{n,j}^{\hat{\sigma}_j} \right] \text{ where } \hat{\lambda}_{n,j}^{\hat{\sigma}_j} = 1,$$

   in which $\hat{\mathcal{Z}}_n$ is a subregion of the partition.

3. **tight** in original space:

   The active region for the lifted state $(\hat{x}_n, \hat{\lambda}_n)$ is:

$$\hat{\mathcal{Z}}_n := \mathcal{Z}_n \cap \prod_j \left[\underline{x}_{n,j}^{\hat{\sigma}_j},\ \overline{x}_{n,j}^{\hat{\sigma}_j} \right] \text{ where } \hat{\lambda}_{n,j}^{\hat{\sigma}_j} = 1,$$

   in which $\hat{\mathcal{Z}}_n$ is a subregion of the partition.

   We recover:

$$\mathcal{L}_m^L(\pi_m^*, \rho_m^*) = \mathrm{co}_{\hat{\mathcal{Z}}_n}(\underline{Q}_m)(\hat{x}_n),$$

   i.e., cut value equals the convex envelope of $\underline{Q}_m(\cdot)$ over $\hat{\mathcal{Z}}_n$.

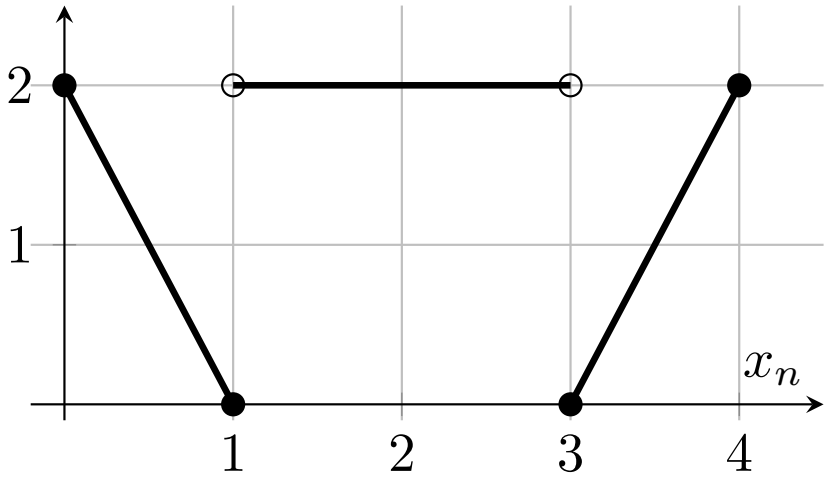
This explains why lifting can achieve recovering the piecewise convex envelope.

Example – Convex Envelope

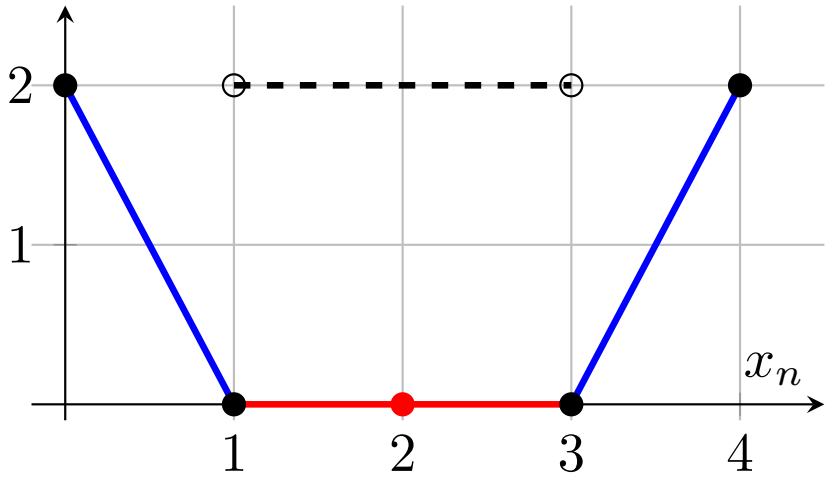
Let us consider the value function

$$Q_m(x_n) = \begin{cases} -2x_n + 2 & \text{if } x_n \in [0,1] \\ 2 & \text{if } x_n \in (1,3) \\ 2x_n - 6 & \text{if } x_n \in [3,4]. \end{cases}$$

Approximation using Lagrangian cuts with $\mathcal{Z}_n = [0,4]$:
(i) tight in the blue regions, (ii) with a gap in the red region.



(a) Value function Q_m



(b) Convex envelope of Q_m over $\mathcal{Z}_n$

Example – Lifting

Since $x_n \in \mathbb{R}$ is continuous over $[0, 4]$, we split $[0, 4]$ at $\hat{x}_n = 2$:



$\mathcal{P}_n = \{[0, 2], [2, 4]\}$.

The lifted state variable is (x_n, λ_n).



$\mathbb{P}_n$: $\lambda_n = 0$ if $x_n \in [0, 2]$;
otherwise, $\lambda_n = 1$.

Example – Lifting

Since $x_n \in \mathbb{R}$ is continuous over $[0, 4]$, we split $[0, 4]$ at $\hat{x}_n = 2$:



$$\mathcal{P}_n = \{[0, 2], [2, 4]\}.$$

The lifted state variable is (x_n, λ_n).



$\mathbb{P}_n$: $\lambda_n = 0$ if $x_n \in [0, 2]$;
otherwise, $\lambda_n = 1$.

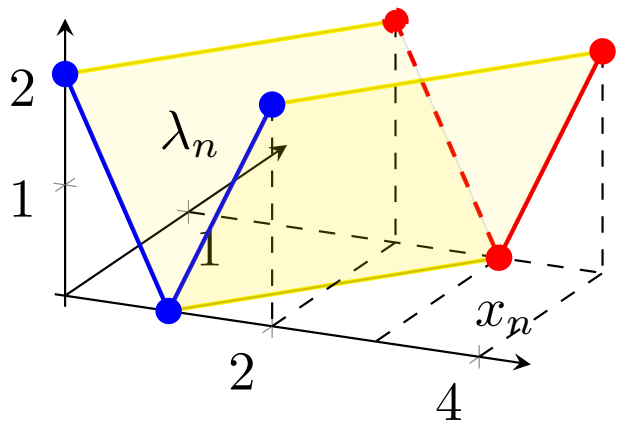
Lifted states $(x_n, \lambda_n) = (2, 1)$ & $(2, 0)$ are extreme points of the set

$$\mathfrak{Z}_n(\mathcal{Z}_n, \mathbb{P}_n) = \left\{ (x_n, \lambda_n) \;\middle|\; \begin{array}{l} \lambda_n \in \{0, 1\} \\ 0 \cdot (1 - \lambda_n) + 2 \cdot \lambda_n \leq x_n \leq 2 \cdot (1 - \lambda_n) + 4 \cdot \lambda_n \end{array} \right\}.$$

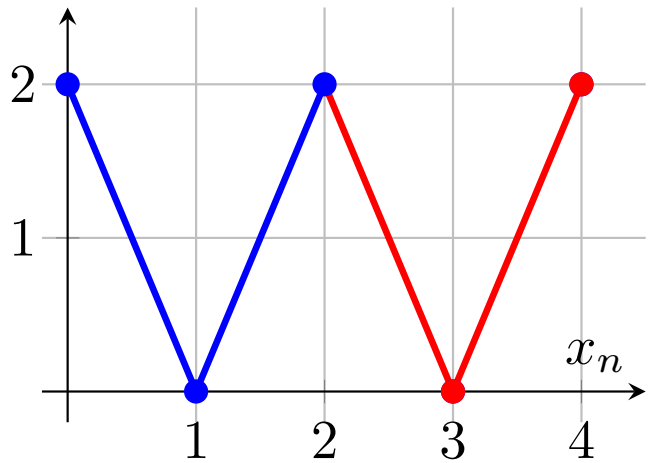
Example – Lifted Convex Envelope

Approximate the value function Q_m via lifted Lagrangian cuts:

$$\theta_m \geq -2x_n + 4\lambda_n + 2,$$
$$\theta_m \geq 2x_n - 4\lambda_n - 2.$$



(c) Convex envelope of in x_n-λ_n space.



(d) Piecewise convex envelope in x_n space.

Figure: Geometric representation of cut approximators.

Stochastic Dual Dynamic Programming with Lifting (SDDP-L) Algorithm

Partition Refinement

Objective: In each iteration, dynamically

- Lift state-variable space;
- Tighten piecewise convex envelope.

Partition Refinement

Objective: In each iteration, dynamically

- Lift state-variable space;
- Tighten piecewise convex envelope.

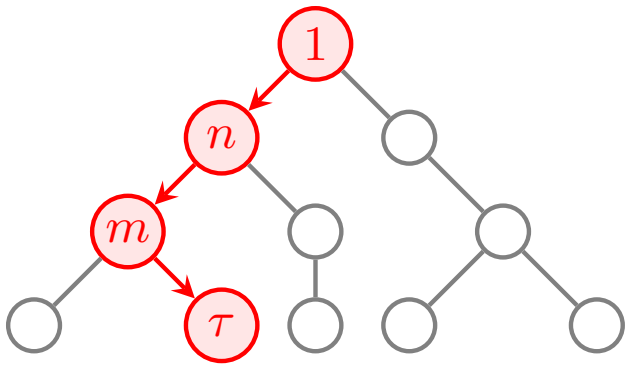
Refinement Ideas: For active interval $[\underline{x}^{\hat{\sigma}}_{n,j}, \overline{x}^{\hat{\sigma}}_{n,j}]$ of component $x_{n,j}$:

- Split at incumbent solution: $[\underline{x}^{\hat{\sigma}}_{n,j}, \hat{x}_{n,j}]$ and $[\hat{x}_{n,j}, \overline{x}^{\hat{\sigma}}_{n,j}]$;
- Split at mid-point: $[\underline{x}^{\hat{\sigma}}_{n,j}, \frac{\underline{x}^{\hat{\sigma}}_{n,j}+\overline{x}^{\hat{\sigma}}_{n,j}}{2}]$ and $[\frac{\underline{x}^{\hat{\sigma}}_{n,j}+\overline{x}^{\hat{\sigma}}_{n,j}}{2}, \overline{x}^{\hat{\sigma}}_{n,j}]$.

SDDP-L – Forward Pass

Forward Pass of Iteration i:

- Sample a scenario path $\mathcal{P}^{i,\omega} = \{1, n, m, \tau\}$;



- **For** each node $n \in \mathcal{P}^{i,\omega}$:

  Solve problem $(P_n^{L,i})$:

$$\begin{aligned} \min \quad & f_n(x_n) + \sum_{m \in \mathcal{C}(n)} q_{nm} \phi_m^{L,i}(x_n, \lambda_n) && (P_n^{L,i}) \\ \text{s.t.} \quad & (x^i_{a(n)}, x_n) \in X_n \\ & \lambda_n \in \Lambda_n(x_n, \mathbb{P}^i_n). \end{aligned}$$

SDDP-L – Backward Pass

Backward Pass of Iteration i:

- **For** each node $n \in \mathcal{P}^{i,\omega}$:

  Refine partition $\mathbb{P}_n^i$ to obtain $\mathbb{P}_n^{i+1}$;

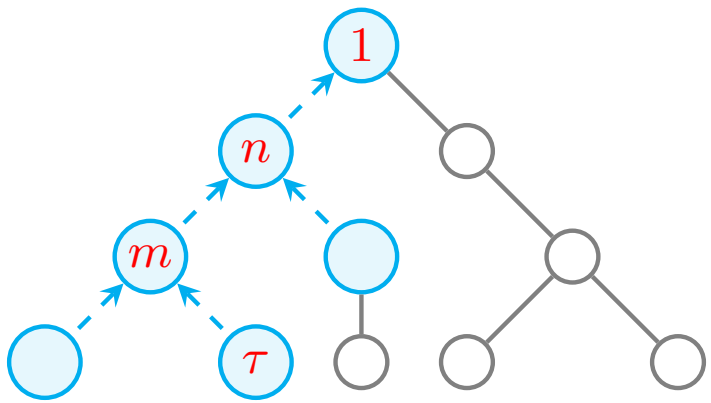
  - **For** each child node $m \in \mathcal{C}(n)$:

    Solve the Lagrangian dual problem $(D_m^{L,i})$

$$\max_{\pi_m, \rho_m} \quad \mathcal{L}_m^L(\pi_m, \rho_m; x_n^i, \lambda_n^i, \Phi_m^{L,i+1}, \mathcal{Z}_n, \mathbb{P}_m^{i+1}, \mathbb{P}_n^{i+1}), \qquad (D_m^{L,i})$$

    to generate a cut of the form

$$\theta_m \geq v_m + \pi_m^\top x_n + \rho_m^\top \lambda_n.$$



Global Convergence

Assumptions: For every node $n \in \mathcal{N}$:

- The size of every element in partition $\mathbb{P}_n^i$ diminishes to 0.
- $\phi_m^{L,i}$ consists of Lagrangian cuts generated at every feasible point.

Global Convergence

Assumptions: For every node $n \in \mathcal{N}$:

- The size of every element in partition $\mathbb{P}_n^i$ diminishes to 0.
- $\phi_m^{L,i}$ consists of Lagrangian cuts generated at every feasible point.

Theorem

For each node $n \in \mathcal{N}$, $\phi_n^i \xrightarrow{e} Q_n$ and $\phi_n^i \xrightarrow{p} Q_n$.

Theorem

Let $\left\{x_1^i\right\}_{i\in\mathbb{N}}$ be a sequence of optimal solutions to the root-node problem (P_n^L). With probability one, we have that:

- *every accumulation point x_1^* of $\left\{x_1^i\right\}_{i\in\mathbb{N}}$ is an optimal solution to model (P);*
- *the lower bounds converge to the optimal value V^*.*

Enhanced Lagrangian Cuts

"Curse" of Extreme Points

Let $\Pi_m(\hat{x}_n)$ be the set of optimal solutions to (D_m).

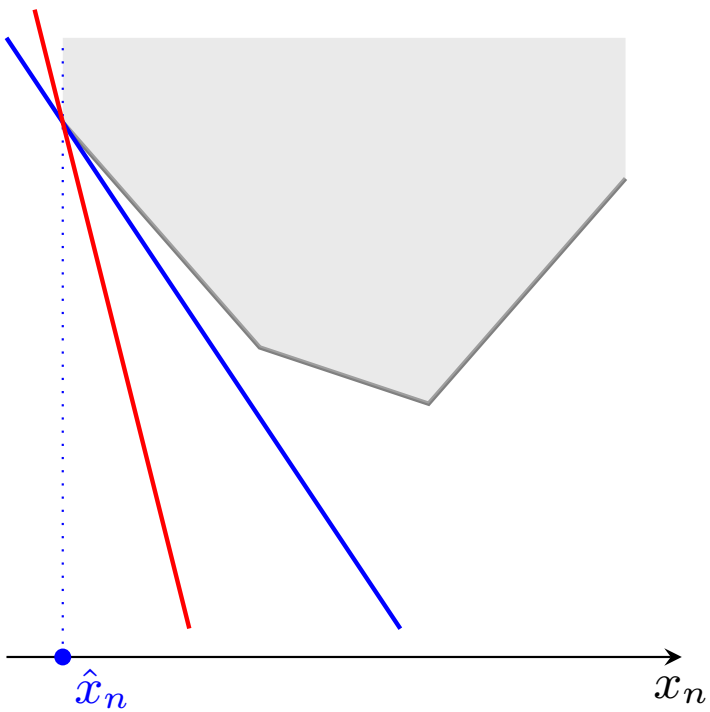
$$\max_{\pi_m} \quad \mathcal{L}_m(\pi_m) \tag{D_m}$$

"Curse" of Extreme Points

Let $\Pi_m(\hat{x}_n)$ be the set of optimal solutions to (D_m).

$$\max_{\pi_m} \quad \mathcal{L}_m(\pi_m) \qquad (D_m)$$

Any $\pi_m \in \Pi_m(\hat{x}_n)$ gives a tight cut:

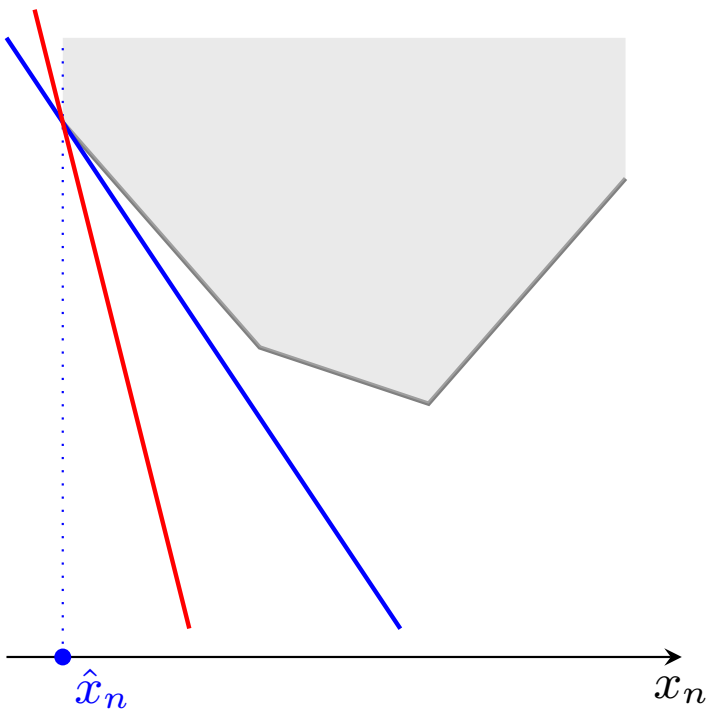


"Curse" of Extreme Points

Let $\Pi_m(\hat{x}_n)$ be the set of optimal solutions to (D_m).

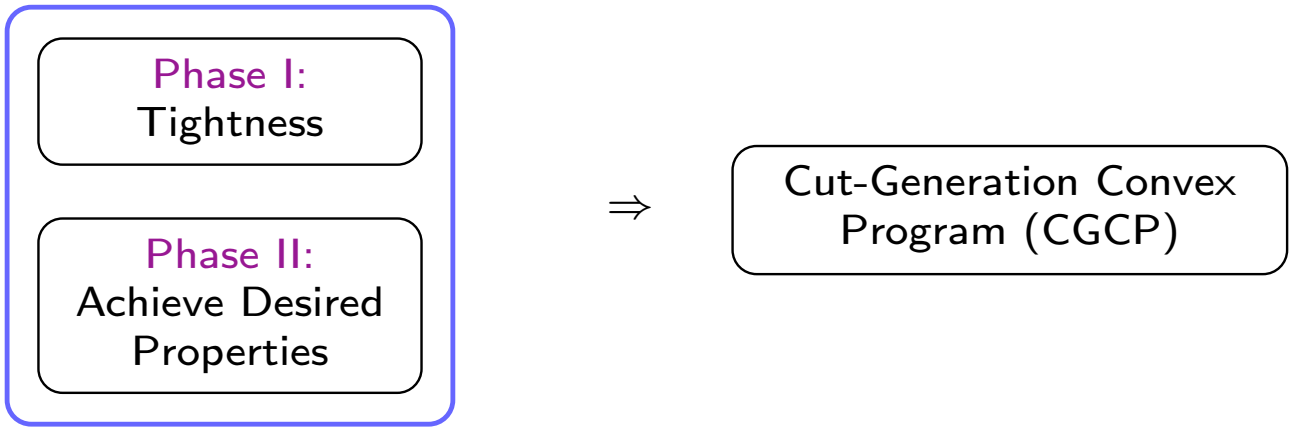
$$\max_{\pi_m} \quad \mathcal{L}_m(\pi_m) \qquad (D_m)$$

Any $\pi_m \in \Pi_m(\hat{x}_n)$ gives a tight cut:

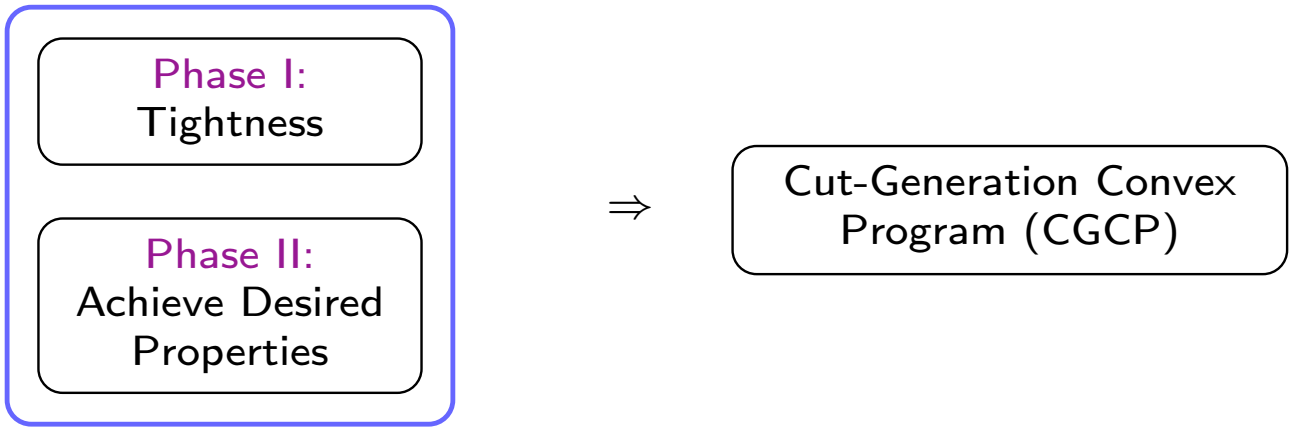


- Some π_m generate cuts with poor approximation away from $\hat{x}_n$;
- Some π_m generate cuts with better approximation beyond $\hat{x}_n$.

Two-Phase Ehancement Method



Two-Phase Ehancement Method

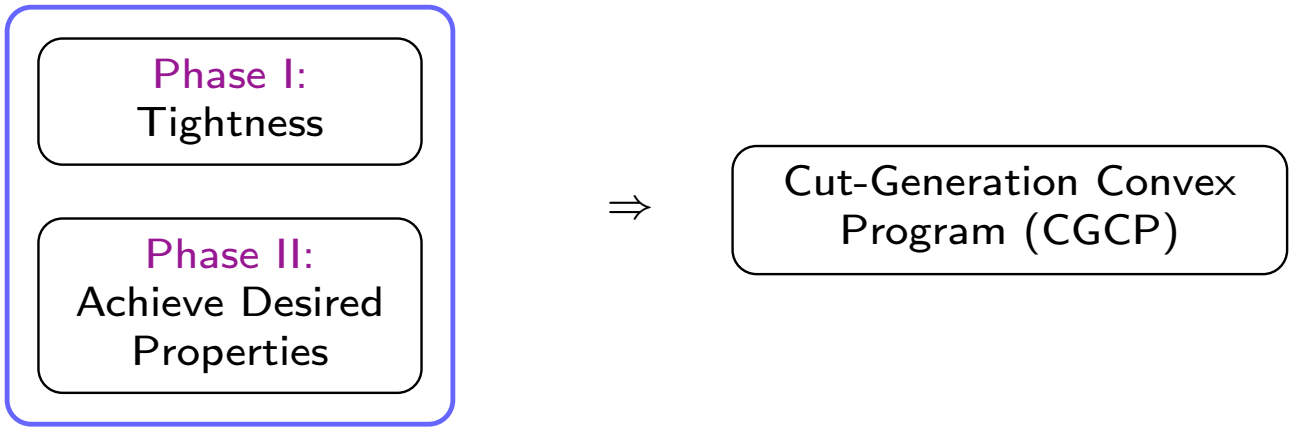


Phase I: Tightness at the Incumbent Point

The **tightness** at $\hat{x}_n$ can be achieve by enforcing:

$$\pi_m \in \Pi_m(\hat{x}_n). \tag{T}$$

Two-Phase Ehancement Method



Phase I: Tightness at the Incumbent Point

The **tightness** at $\hat{x}_n$ can be achieve by enforcing:

$$\pi_m \in \Pi_m(\hat{x}_n). \tag{T}$$

Convex relaxation: *Given* $\hat{\pi}_m \in \Pi_m(\hat{x}_n)$, and a small tolerance $\epsilon > 0$.

To achieve convexity, replace (T) above with

$$\mathcal{L}_m(\pi_m) \geq \mathcal{L}_m(\hat{\pi}_m) - \epsilon.$$

Pareto-optimal Lagrangian Cut (PLC)

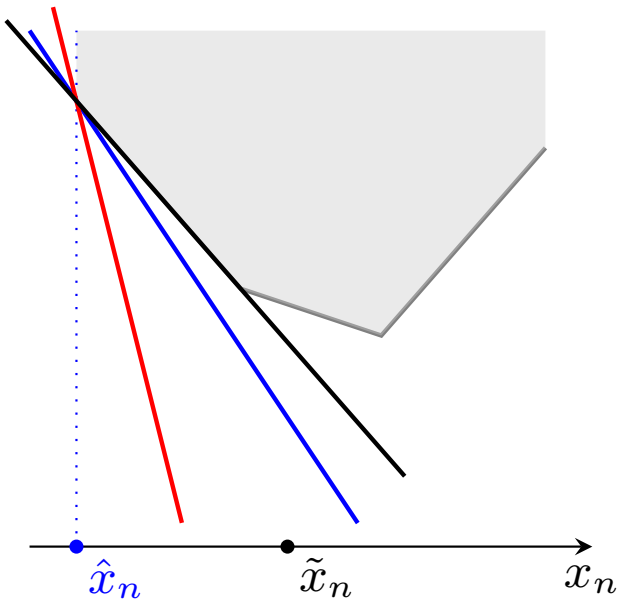
Phase II: Pareto-optimality

Phase II: Improve Cut Values at Core Point

Let $\tilde{x}_n \in \text{relint}(\text{conv}(\mathcal{Z}_n))$ be a core point.

- **Maximize** its cut value at $\tilde{x}_n$:

$$\max \quad \mathcal{L}_m\left(\pi_m; \tilde{x}_n, \Phi_m, \mathcal{Z}_n\right)$$



Pareto-optimal Lagrangian Cut (PLC)

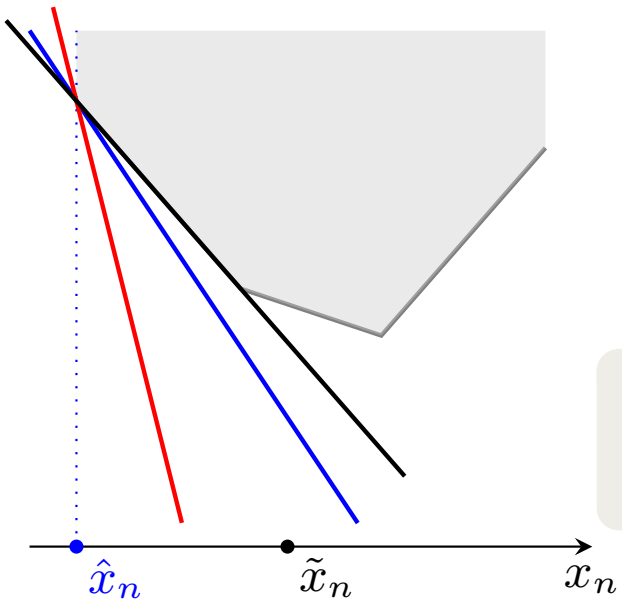
Phase II: Pareto-optimality

Phase II: Improve Cut Values at Core Point

Let $\tilde{x}_n \in \text{relint}(\text{conv}(\mathcal{Z}_n))$ be a core point.

- **Maximize** its cut value at $\tilde{x}_n$:

$$\max \quad \mathcal{L}_m\left(\pi_m; \tilde{x}_n, \Phi_m, \mathcal{Z}_n\right)$$



$$\begin{aligned} \max_{\pi_m} \quad & \mathcal{L}_m\left(\pi_m; \hat{x}_n, \Phi_m, \mathcal{Z}_n\right) + \pi_m^\top(\tilde{x}_n - \hat{x}_n) \\ \text{s.t.} \quad & \mathcal{L}_m\left(\pi_m\right) \geq \\ & \quad \mathcal{L}_m\left(\hat{\pi}_m\right) - \epsilon. \end{aligned}$$

PLC is Pareto-optimal.

Square-Minimization Cut (SMC)

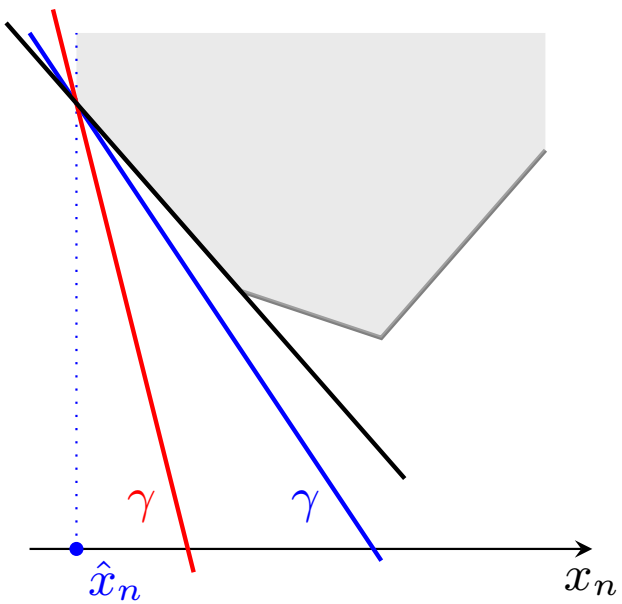
Phase II: Flatness

Phase II: Generate "Flat" Cuts

Let γ be the angle formed by the cut and the horizontal axis.

- **Maximize** cosine of the angle γ:

$$\max \quad \cos(\gamma) := \frac{1}{\sqrt{{\pi_m}^\top \pi_m + 1}}$$



Square-Minimization Cut (SMC)

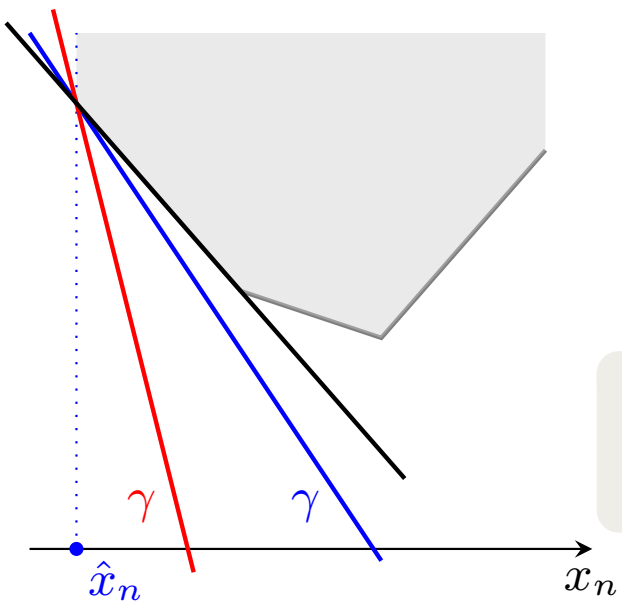
Phase II: Flatness

Phase II: Generate "Flat" Cuts

Let γ be the angle formed by the cut and the horizontal axis.

- **Maximize** cosine of the angle γ:

$$\max \quad \cos(\gamma) := \frac{1}{\sqrt{{\pi_m}^\top \pi_m + 1}}$$



$$\begin{aligned} \min_{\pi_m} \quad & \pi_m^\top \pi_m \\ \text{s.t.} \quad & \mathcal{L}_m(\pi_m) \geq \\ & \mathcal{L}_m(\hat{\pi}_m) - \epsilon. \end{aligned}$$

SMC avoids selecting a core point.

Numerical Experiments

Multistage Stochastic Unit Commitment (MSUC)

- MSUC with power flow, power balance, ramping, and minimum-up/-down constraints to minimize production and load-shedding costs.

$$\min \quad \sum_{t\in\mathcal{T}} \left[\sum_{g\in\mathcal{G}} \left(h_{gt} + c_g^{Start} v_{gt} + c_g^{Down} w_{gt} \right) + \sum_{d\in\mathcal{D}} w_d (1 - x_{dt}) \right]$$

$$\text{s.t.} \quad h_{gt} \geq c_{og}^1 P_{gt}^G + c_{og}^0 y_{gt} \qquad \forall o \in \mathcal{O}, g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{ijt}^L = B_{ij} \left(\theta_{it} - \theta_{jt} \right) \qquad \forall (i,j) \in \mathcal{L}, t \in \mathcal{T}$$

$$-W_{ij} \leq P_{ijt}^L \leq W_{ij} \qquad \forall (i,j) \in \mathcal{L}$$

$$\underline{P}_g^G y_{gt} \leq P_{gt}^G \leq \overline{P}_g^G y_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$\sum_{g\in\mathcal{G}_i} P_{gt}^G + \sum_{(i,j)\in\mathcal{L}} P_{ijt}^L = \sum_{d\in\mathcal{D}_i} D_{dt} x_{dt} \qquad \forall i \in \mathcal{B}, t \in \mathcal{T}$$

$$v_{gt} - w_{gt} = y_{gt} - y_{g,t-1} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{gt}^G - P_{g,t-1}^G \leq M_g y_{g,t-1} + \underline{P}_g^G v_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{g,t-1}^G - P_{gt}^G \leq M_g y_{gt} + \underline{P}_g^G w_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$v_{gt}, y_{gt}, w_{gt} \in \{0,1\} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$x_{dt} \in [0,1] \qquad \forall d \in \mathcal{D}, t \in \mathcal{T}$$

Multistage Stochastic Unit Commitment (MSUC)

- MSUC with power flow, power balance, ramping, and minimum-up/-down constraints to minimize production and load-shedding costs.

$$\min \quad \sum_{t\in\mathcal{T}} \left[\sum_{g\in\mathcal{G}} \left(h_{gt} + c_g^{Start} v_{gt} + c_g^{Down} w_{gt}\right) + \sum_{d\in\mathcal{D}} w_d(1 - x_{dt}) \right]$$

$$\text{s.t.} \quad h_{gt} \geq c_{og}^1 P_{gt}^G + c_{og}^0 y_{gt} \qquad \forall o \in \mathcal{O}, g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{ijt}^L = B_{ij}\left(\theta_{it} - \theta_{jt}\right) \qquad \forall (i,j) \in \mathcal{L}, t \in \mathcal{T}$$

$$-W_{ij} \leq P_{ijt}^L \leq W_{ij} \qquad \forall (i,j) \in \mathcal{L}$$

$$\underline{P}_g^G y_{gt} \leq P_{gt}^G \leq \overline{P}_g^G y_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$\sum_{g\in\mathcal{G}_i} P_{gt}^G + \sum_{(i,j)\in\mathcal{L}} P_{ijt}^L = \sum_{d\in\mathcal{D}_i} D_{dt} x_{dt} \qquad \forall i \in \mathcal{B}, t \in \mathcal{T}$$

$$v_{gt} - w_{gt} = y_{gt} - y_{g,t-1} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{gt}^G - P_{g,t-1}^G \leq M_g y_{g,t-1} + \underline{P}_g^G v_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$P_{g,t-1}^G - P_{gt}^G \leq M_g y_{gt} + \underline{P}_g^G w_{gt} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$v_{gt}, y_{gt}, w_{gt} \in \{0,1\} \qquad \forall g \in \mathcal{G}, t \in \mathcal{T}$$

$$x_{dt} \in [0,1] \qquad \forall d \in \mathcal{D}, t \in \mathcal{T}$$

- **State Variables**: Binary v, y, w & Continuous P^G.

Experiment – Setup

We consider three algorithms

- SDDP-L: SDDP + Lifted Lagrangian cuts;
- SDDiP: binarize all non-binary variables $\rightarrow$ Approximation;
- SDDP: Lagrangian cuts.

Experiment – Setup

We consider three algorithms

- SDDP-L: SDDP + Lifted Lagrangian cuts;
- SDDiP: binarize all non-binary variables $\rightarrow$ Approximation;
- SDDP: Lagrangian cuts.

With the following setups:

- Use IEEE case30 [Shahidehpour and Wang, 2003];
- Random variables: the demand D_{dt};
- CGCPs are solved by using the level method [Lemaréchal et al., 1995];

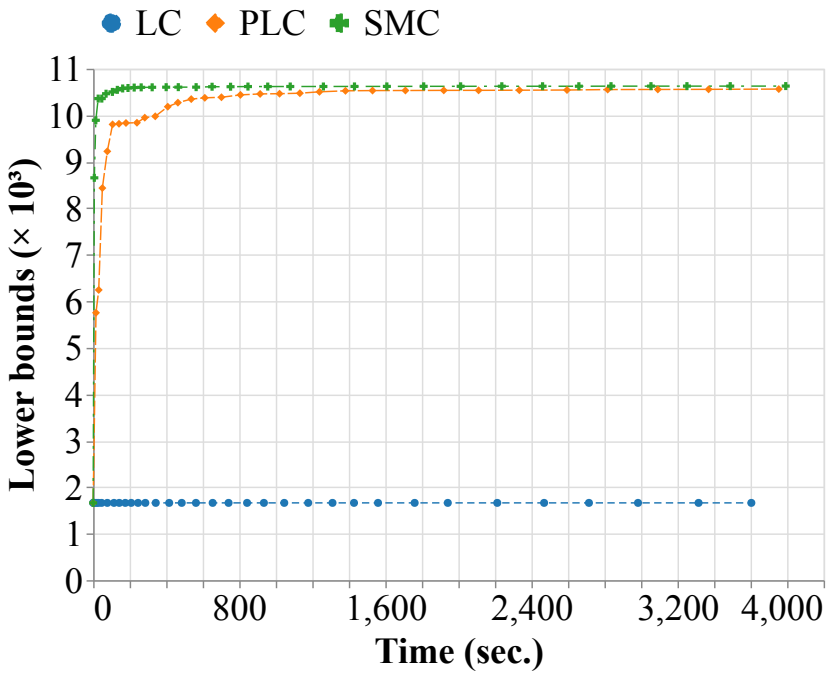
$\rightsquigarrow$ Time Limit $= 3,600$s.

SDDP-L – Incumbent Solutions

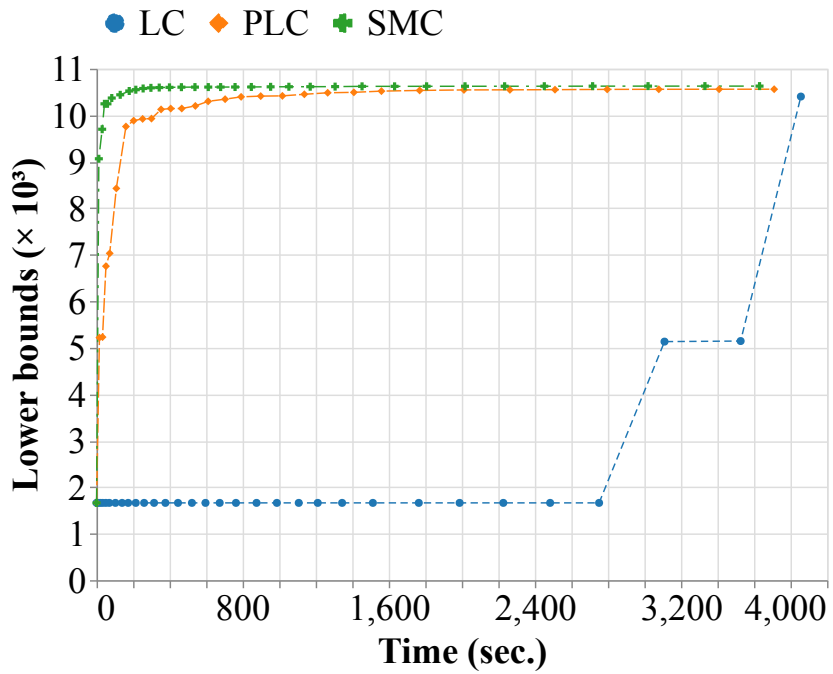
SDDP-L Algorithm with Partitions Refined according to Incumbent Solutions								
Cut	T	R	LB	Gap(%)	Iter.	Time/Iter.	$\text{Time}_{1\%}$	$\text{Iter}_{1\%}$
LC	6	5	1629.5	61.3	52	73.4 (54.1)	-	-
		10	1296.2	66.4	51	74.5 (55.9)	-	-
	8	5	1662.8	74.1	44	88.3 (68.4)	-	-
		10	1662.8	74.2	43	88.8 (70.1)	-	-
	12	5	1662.8	84.5	33	112.6 (94.0)	-	-
		10	1999.7	83.2	45	85.4 (44.9)	-	-
PLC	6	5	4203.6	0.1	56	66.3 (55.0)	24.3	4
		10	3849.0	0.2	48	79.2 (57.0)	33.1	4
	8	5	6405.7	0.3	42	88.0 (70.9)	85.7	8
		10	6386.8	0.6	39	96.1 (62.5)	290.1	10
	12	5	10563.1	1.0	32	121.1 (94.3)	1709.9	24
		10	11574.1	2.5	30	133.4 (89.1)	-	-
SMC	6	5	4206.3	0.1	57	66.0 (41.4)	16.6	4
		10	3853.4	0.0	53	71.5 (49.4)	17.3	4
	8	5	6413.1	0.1	44	87.9 (60.8)	46.4	6
		10	6399.3	0.0	43	88.8 (63.6)	71.5	7
	12	5	10627.8	0.1	34	114.9 (84.8)	161.7	9
		10	11749.2	0.5	36	104.8 (80.4)	308.3	12

SDDP-L – Bisection

SDDP-L Algorithm with Partitions Refined according to Bisection								
Cut	T	R	LB	Gap(%)	Iter.	Time/Iter.	$\text{Time}_{1\%}$	$\text{Iter}_{1\%}$
LC	6	5	4137.5	1.7	50	77.2 (60.6)	-	-
		10	3788.2	1.7	48	77.5 (60.5)	-	-
	8	5	6342.2	0.9	42	91.8 (80.9)	2082.2	35
		10	4004.9	37.4	41	94.5 (81.5)	-	-
	12	5	10402.3	10.8	32	124.4 (112.7)	-	-
		10	1999.7	83.0	32	121.0 (94.0)	-	-
PLC	6	5	4203.4	0.1	49	76.2 (55.4)	30.7	4
		10	3847.6	0.2	46	83.2 (55.8)	30.2	4
	8	5	6408.8	0.4	41	92.1 (76.3)	190.6	10
		10	6389.5	0.2	39	101.5 (78.0)	146.0	8
	12	5	10560.3	1.0	31	123.7 (97.8)	2796.1	28
		10	11568.1	1.9	28	133.6 (99.6)	-	-
SMC	6	5	4206.4	0.1	49	75.0 (64.8)	18.0	4
		10	3853.1	0.1	50	74.3 (59.2)	18.5	4
	8	5	6413.7	0.3	42	91.6 (75.1)	73.9	7
		10	6399.2	0.5	41	91.2 (78.7)	129.1	9
	12	5	10627.1	0.0	33	113.4 (92.1)	254.9	10
		10	11746.0	0.8	32	117.7 (96.2)	256.7	10



(a) $R = 5$, SDDP-L, refining partition according to incumbent solutions.



(b) $R = 5$, SDDP-L, refining partition according to bisection.

Figure: Progression of lower bounds for the MSUC instance with T = 12.

Benchmark vs. Binarization

SDDiP Algorithm with State Variables Binary Representation of Precision 2^{-8}								
Cut	T	R	LB	Gap(%)	Iter.	Time/Iter.	$\text{Time}_{1\%}$	$\text{Iter}_{1\%}$
LC	6	5	2037.9	54.6	30	129.5 (67.3)	-	-
		10	1554.4	61.7	32	123.3 (77.0)	-	-
	8	5	2003.7	69.5	29	138.0 (72.2)	-	-
		10	2003.4	69.8	25	159.0 (90.1)	-	-
	12	5	1952.8	82.5	24	160.6 (63.2)	-	-
		10	2395.9	80.4	22	180.4 (72.9)	-	-
PLC	6	5	2832.3	35.0	7	704.5 (114.2)	-	-
		10	2391.1	41.5	5	983.4 (125.7)	-	-
	8	5	2483.1	64.4	6	866.2 (127.0)	-	-
		10	2275.5	65.6	4	1569.0 (322.4)	-	-
	12	5	2271.1	79.4	4	1217.3 (35.9)	-	-
		10	2705.5	78.3	3	1900.9 (497.8)	-	-
SMC	6	5	4266.1	0.1	60	62.5 (29.0)	120.4	5
		10	3887.9	0.2	30	127.2 (48.9)	349.6	5
	8	5	5354.9	18.2	23	171.2 (55.7)	-	-
		10	5437.8	16.6	23	173.4 (76.3)	-	-
	12	5	10041.1	8.4	21	193.0 (68.7)	-	-
		10	10985.2	8.5	27	149.0 (82.6)	-	-

Duality Gap Closed via Lifting

SDDP with Lagrangian Cuts (Algorithm 1)						
Cut	T	R	LB	Gap(%)	Iter.	Time/Iter.
LC	6	5	1629.5	61.3	116	31.4 (24.3)
		10	1296.2	66.4	108	34.3 (21.5)
	8	5	1662.8	74.1	94	39.9 (30.0)
		10	1662.8	74.1	97	37.5 (26.0)
	12	5	1662.8	84.4	72	51.9 (44.6)
		10	1999.7	83.1	76	49.2 (32.1)
PLC	6	5	4169.3	0.9	108	34.4 (25.2)
		10	3819.0	0.9	89	41.1 (22.7)
	8	5	6327.1	1.7	89	41.6 (28.9)
		10	6277.8	2.1	82	45.6 (28.2)
	12	5	10476.0	2.0	65	56.5 (44.0)
		10	11574.4	1.5	62	59.2 (34.6)
SMC	6	5	4172.7	0.9	131	27.8 (21.2)
		10	3822.6	0.8	104	35.8 (29.8)
	8	5	6330.3	1.6	104	35.0 (25.3)
		10	6265.6	2.0	94	39.2 (27.9)
	12	5	10490.3	1.7	80	46.8 (39.0)
		10	11571.9	1.9	70	52.9 (45.3)

Outline

1 Introduction

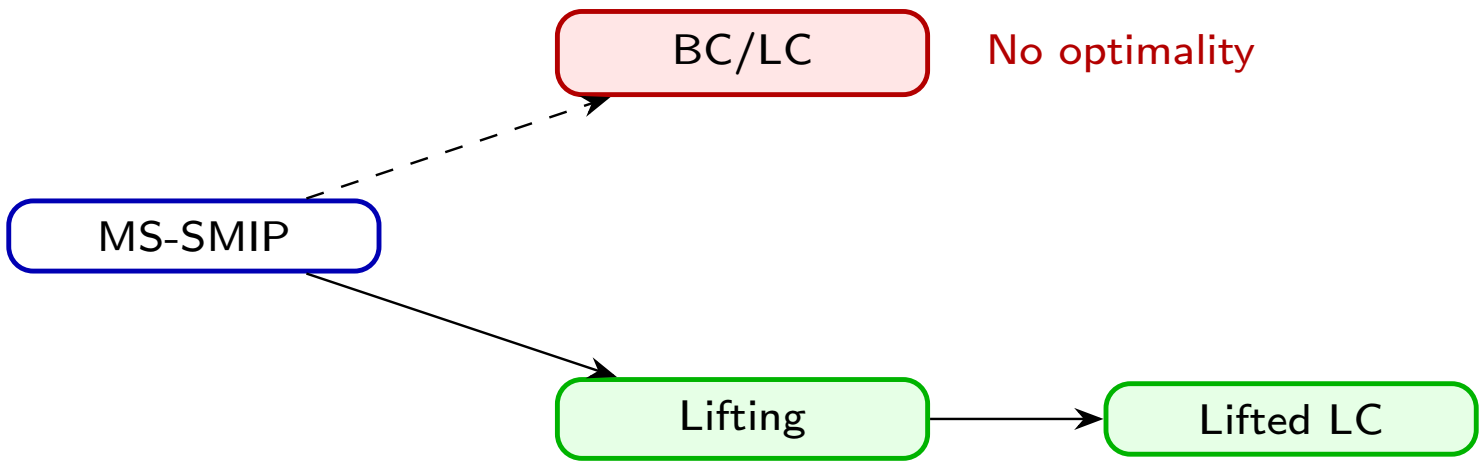
2 Lift-and-Cut Algorithm

3 Cut Enhancement

4 Numerical Experiments

5 Summary

- **Lift-and-Cut Algorithm for MS-SMIP:**
  - Cut Tightness by lifting;
  - Convergence of cut approximators;
  - Guarantees global convergence of SDDP-L.
- **Enhanced Lagrangian Cuts:**
  - Pareto-optimality of PLC;
  - Heuristic method for generating SMC;
  - Significant numerical improvements.



Acknowledgements

Thank you for attending!

hanbinyang@ust.hk

References I

C. Lemaréchal, A. Nemirovskii, and Y. Nesterov. New variants of bundle methods. *Mathematical programming*, 69(1):111–147, 1995.

Mohammad Shahidehpour and Yaoyu Wang. *Appendix C: IEEE30 Bus System Data*, pages 493–495. 2003. doi: 10.1002/0471462926.app3.